

Analizador de respuesta en frecuencia.

Matias Lapietra, Ariel Monffyllo

Departamento de Ingeniería Electrónica, Universidad Tecnológica Nacional. Facultad Regional Avellaneda
Buenos Aires, Argentina

matias.e.lapietra@gmail.com

ariel.monffyllo@gmail.com

Abstract— El proyecto realizado consiste en un analizador de respuesta en frecuencia capaz de caracterizar la respuesta en amplitud de un circuito o sistema externo mediante un barrido en frecuencia. El sistema genera una señal senoidal de amplitud fija mediante un DDS, aplica dicho estímulo al sistema bajo prueba y mide las amplitudes de las señales de entrada y salida a través de detectores de pico y el conversor analógico-digital (ADC) interno del microcontrolador. A partir de estas mediciones calcula la ganancia en dB para cada punto y construye el diagrama de Bode, que se visualiza en tiempo real sobre una pantalla a medida que avanza el barrido. El usuario puede configurar la frecuencia de inicio, la frecuencia final, la cantidad de puntos y el tiempo de barrido. Adicionalmente, se implementa la exportación de tramas por UART. Se desarrolla con un microcontrolador ESP32-S3 (arquitectura Xtensa LX7) sobre la placa Waveshare ESP32-S3-Touch-LCD-2, utilizando un sistema operativo en tiempo real (FreeRTOS). Se programa en lenguaje C con ESP-IDF v6.0.

Para generar el estímulo de excitación del sistema bajo prueba se empleó el generador de forma de onda programable AD9833[1], un DDS capaz de producir señales senoidales con frecuencia configurable por software a través de una interfaz SPI. Este componente entrega una senoidal de amplitud fija cuya frecuencia se actualiza en cada punto del barrido, cumpliendo así con el requisito de estímulo del sistema.

La etapa de adquisición debe medir la amplitud de las señales de entrada y de salida del sistema bajo prueba. Dado que estas señales son alternas y que el ADC interno del microcontrolador mide niveles de tensión continua, se incorporan detectores de pico por hardware que convierten la amplitud de cada senoidal en un nivel de continua. A partir de las amplitudes de entrada (Vin) y salida (Vout) se calcula la ganancia del sistema en dB para cada frecuencia.

La información del sistema se visualiza a través de una pantalla de 320×240 píxeles integrada en la placa Waveshare ESP32-S3-Touch-LCD-2, con el controlador ST7796 y un controlador táctil capacitivo CST816D. La interfaz de usuario permite configurar los parámetros del barrido mediante un encoder rotativo y un pulsador. Para la exportación de datos hacia una PC se utiliza la interfaz UART.

I. ÍNDICE

I. ÍNDICE	1
II. FUNDAMENTACIÓN	1
A. Problemática	1
B. Fundamentación Teórica de Hardware	1
C. Fundamentación Teórica de Software	1
III. DIAGRAMA EN BLOQUES	2
IV. DIAGRAMA DE TAREAS	2
REFERENCIAS	2

II. FUNDAMENTACIÓN

A. Problemática

La caracterización de la respuesta en frecuencia de un circuito es una tarea recurrente en el análisis y diseño electrónico, ya que permite conocer cómo varía su ganancia en función de la frecuencia de la señal de entrada. Determinar manualmente la ganancia en frecuencia resulta poco práctico y propenso a errores cuando se requiere una cantidad elevada de mediciones. Se hace necesario, entonces, un sistema que automatice el barrido en frecuencia, realice la adquisición de las amplitudes de entrada y salida, y presente el resultado de forma directa como un diagrama de Bode, reduciendo el tiempo de medición y mejorando la repetibilidad del proceso.

B. Fundamentación Teórica de Hardware

C. Fundamentación Teórica de Software

El proyecto ha sido desarrollado sobre un sistema operativo en tiempo real, específicamente FreeRTOS, lo cual implica la utilización de tareas concurrentes coordinadas mediante mecanismos de sincronización como colas, semáforos y mutex. La arquitectura se organiza en tareas independientes que se comunican exclusivamente a través de estos mecanismos.

La tarea encargada del barrido controla el AD9833 y, para cada punto de frecuencia, dispara la generación del estímulo y la medición de las amplitudes mediante el ADC interno. Los resultados obtenidos se transmiten hacia la tarea de visualización a través de una cola, que los recibe y actualiza el diagrama de Bode en pantalla utilizando la biblioteca gráfica LVGL. La gestión de la interfaz de usuario se realiza en una tarea dedicada que comunica los eventos al resto del sistema a través de colas.

La depuración del sistema se realiza mediante el envío de logs a través de la interfaz USB nativa del ESP32-S3, mientras que la exportación de tramas de datos hacia la PC se efectúa por la UART, separando así ambos canales. UART constituye un recurso compartido del que solo puede hacer uso una tarea a la vez en transmisión; por ello, su acceso se protege mediante un semáforo del tipo mutex. La configuración del usuario se almacena en la memoria no volátil (NVS) del microcontrolador, gestionada por una tarea específica.

III. DIAGRAMA EN BLOQUES

Los módulos que componen el sistema se muestran en la figura 1, donde se pueden observar las conexiones entre ellos y a qué pines corresponden.

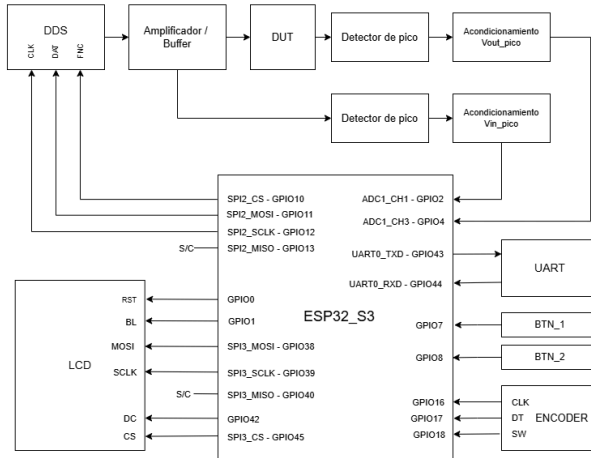


Fig. 1 Diagrama en bloques del analizador de respuesta en frecuencia

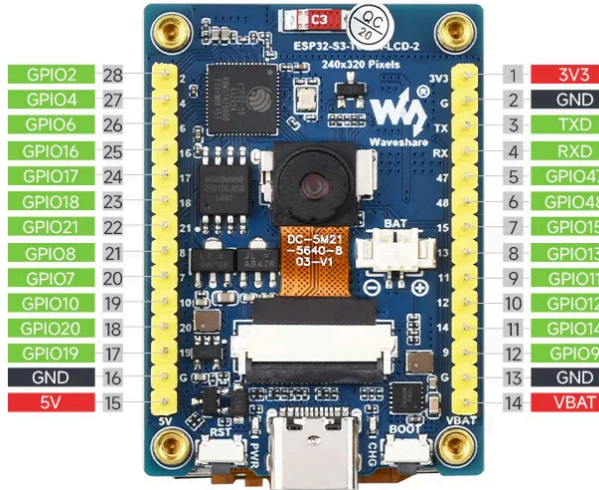
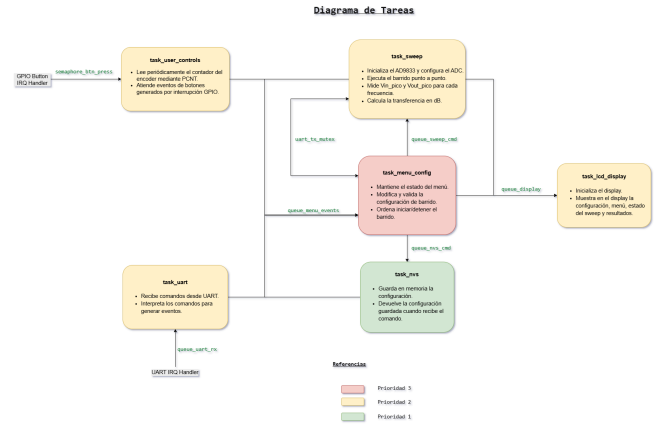


Fig. 2 Pinout placa Waveshare ESP32-S3-Touch-LCD-2

IV DIAGRAMA DE TAREAS

A continuación un detalle de las tareas que se ejecutarán en el ESP-32 S3.



Estructura del Firmware:

El firmware implementa un sistema multitarea sobre FreeRTOS en el microcontrolador ESP32-S3, organizado en módulos independientes que se comunican mediante **colas** y **semáforos**. Cada tarea tiene una prioridad definida según su criticidad.

◆ Organización de tareas

GPIO Button IRQ Handler (Prioridad alta)

- Atiende interrupciones de botones y del encoder. Señaliza eventos mediante el semáforo `sem_btn_press` para que la tarea de controles de usuario los procese.

task_user_controls (Prioridad 2)

- Lee periódicamente el estado del encoder y botones. Genera eventos de usuario que se envían a `queue_menu_events`.

task_menu_config (Prioridad 2)

- Mantiene y valida la configuración del barrido de frecuencias. Recibe eventos desde el menú y puede ordenar iniciar/detener el sweep. Envía mensajes a `queue_nvs_cmd` para guardar configuración y a `queue_display` para actualizar la interfaz.

task_sweep (Prioridad 3)

- Inicializa el generador DDS (AD9833) y el ADC. Ejecuta el barrido punto a punto, midiendo Vin y Vout. Calcula la transferencia en dB y envía resultados a través de `queue_sweep_cmd`.

task_uart (Prioridad 2)

- Recibe comandos desde el PC a través de **UART0** y los interpreta para generar eventos. El envío de datos por UART está protegido con el **mutex** `mutex_uart_tx`, garantizando acceso exclusivo y evitando colisiones

entre tareas.

task_nvs (Prioridad 2)

- Gestiona el almacenamiento en memoria flash (NVS). Guarda configuraciones recibidas y responde consultas devolviendo valores almacenados. Se comunica mediante `queue_nvs_cmd`.

task_lcd_display (Prioridad 2)

- Inicializa el display y la librería gráfica LVGL. Muestra en pantalla la configuración vigente, el estado del barrido y los resultados. Recibe mensajes desde `queue_display`.

UART IRQ Handler

- Atiende interrupciones de recepción en UART0 y coloca los datos en `queue_uart_rx`.

♦ Comunicación y sincronización

Colas (Queues):

- Se utilizan para pasar mensajes entre tareas (ej. configuración, resultados de barrido, datos para mostrar en LCD).
- **Semáforos (Semaphores):**
 - `sem_btn_press`: sincroniza eventos de botones desde la IRQ GPIO.
 - `mutex_uart_tx`: protege el acceso concurrente al periférico UART0.
- **Interrupciones (IRQ):**
 - Botones y encoder generan eventos que se capturan por GPIO.
 - UART0 genera interrupciones de recepción que alimentan la cola `queue_uart_rx`.

♦ Flujo general

1. El usuario interactúa con botones → IRQ → `task_user_controls`.
2. `task_menu_config` válida parámetros y actualiza configuración.
3. `task_sweep` ejecuta el barrido y envía resultados.
4. `task_lcd_display` muestra en pantalla los valores confirmados y resultados.
5. `task_uart` comunica con el PC vía UART0, protegido por mutex.
6. `task_nvs` guarda y recupera configuraciones en memoria flash.

En conclusión, el firmware implementa un sistema modular y concurrente donde cada tarea cumple una función específica, coordinada mediante colas y semáforos de FreeRTOS. La comunicación con el PC se realiza por UART0 con acceso protegido por mutex, y los eventos de usuario y display se gestionan mediante interrupciones, garantizando respuesta rápida y confiable.

REFERENCIAS

- Analog Devices, Low Power, 12.65 mW, 2.3 V to 5.5 V, Programmable Waveform Generator (AD9833), Data Sheet Rev. G. Norwood, MA, EE.UU.: Analog Devices, Inc., 2019.
- Espressif Systems, *ESP32-S3 Series Datasheet*. [2] Shanghai, China: Espressif Systems, 2023.
- Waveshare, ESP32-S3-Touch-LCD-2 Wiki. [En línea]. [3] Disponible: <https://www.waveshare.com/wiki/ESP32-S3-Touch-LCD-2>
- LVGL, LVGL Documentation, v9. [En línea]. Disponible: [4] <https://docs.lvgl.io/>